

International University of Sarajevo

Faculty of Engineering and Natural Sciences

Database Management

CS306 Spring 2025

PROJECT 1 REPORT

Airport Database Management System

Students: Berina Juković, Hena Šehović, Berin Žunić

Spring,2025.

Contents

1. Introduction 2

2. Defining the Database 4
2.1 Applying Three-level Architecture 4
2.2 Identify the purpose of the DBMS 7
2.3 Determining the entities, types of attributes and relationships involved 8
2.3.1 Entities 8
2.5.2 Attributes 8
2.2.3 Relationships 12
2.2.4 Constraints 13
2.3 Queries 14
3 Conclusion 18
4 Resources 19
5 List of Figures 19

1. Introduction

Airport as an idea has existed since humans first flew, with the first recorded use of an airport being in 1909 in Maryland, USA. That airport was established as a training ground for military pilots. During that period military and government were only airport users. Mentioned airport later became a civilian airport and hosted the first scheduled commercial flight in 1926. The

modern airports emerged in the 20th century due to advancements in aviation technology, allowing planes to fly farther and carry more passengers.

Today, airports are mostly used for commercial and civilian purposes, with the growth of commercial aviation in the early 20th century. Modernization and expansion have included additional terminal buildings, longer runways, and a wider range of facilities and services.

Airports serve as an important center for global connections, transportation of both people and goods. They are one of the most complicated and dynamic infrastructures both in the transportation sector and in the world generally. As such they require good systems to control them and manage large amounts of data needed to run such a structure.

Airport operations were initially manually managed using physical logbooks, paper records, and simple administrative tools. These aproaches were certainly not productive, prone to human error, and unable to keep up with rising demands as passenger traffic and airport complexity grew. The first change began with the introduction of computers in the mid-20th century, making it possible to use simple record-keeping and reservations systems.

Relational database management systems (RDBMS) started to become essential to airline operations by the 1980s and 1990s. These tecnologies made it possible for data to be effectively updated, queried, and saved across numerous terminals and users. The demand for reliable, centralized, and scalable database systems increased with the growth of online ticketing, real-time flight tracking, and international airline alliances.

These days, airport DBMSs also coordinate personnel, track maintenance, and security systems. They are also connected to other systems including border control platforms, weather services, and airline databases. This historical development emphasizes how important database systems are to preserve aviation sector dependability, efficiency, and safety.

From allocating gates and scheduling flights to overseeing passengers, crew, and baggage, running a modern airport requires coordinating a wide range of complicated tasks. The volume and complexity of air travel are only going to increase, making an organized and effective database system crucial. Through the ogranization of data into clearly defined entities, including airports, airplanes, flights, passengers, tickets, and employees, the system offers a centralized and dependable platform for monitoring and controlling every part of airport operations. The total experience for both passengers and employees is improved, in addition to operational efficiency and safety.

This project is a relational database that is meant to manage various aspects of airport operation. MySQL, an open-source relational database management system (RDBMS), was used in designing this database project. Because of its dependablity, resilience, and

compatibility with a wide range of development environments and tools, we selected MySQL. It is ideal complex, practical applications like an airport system because it has effective storage engines, transaction support, and a strong focus on data integrity. A local MySQL server served as the development environment, and MySQL 8.0 was used to implement all schema design, table creation, and data integrity restrictions.

2. Defining the Database

2.1 Applying Three-level Architecture

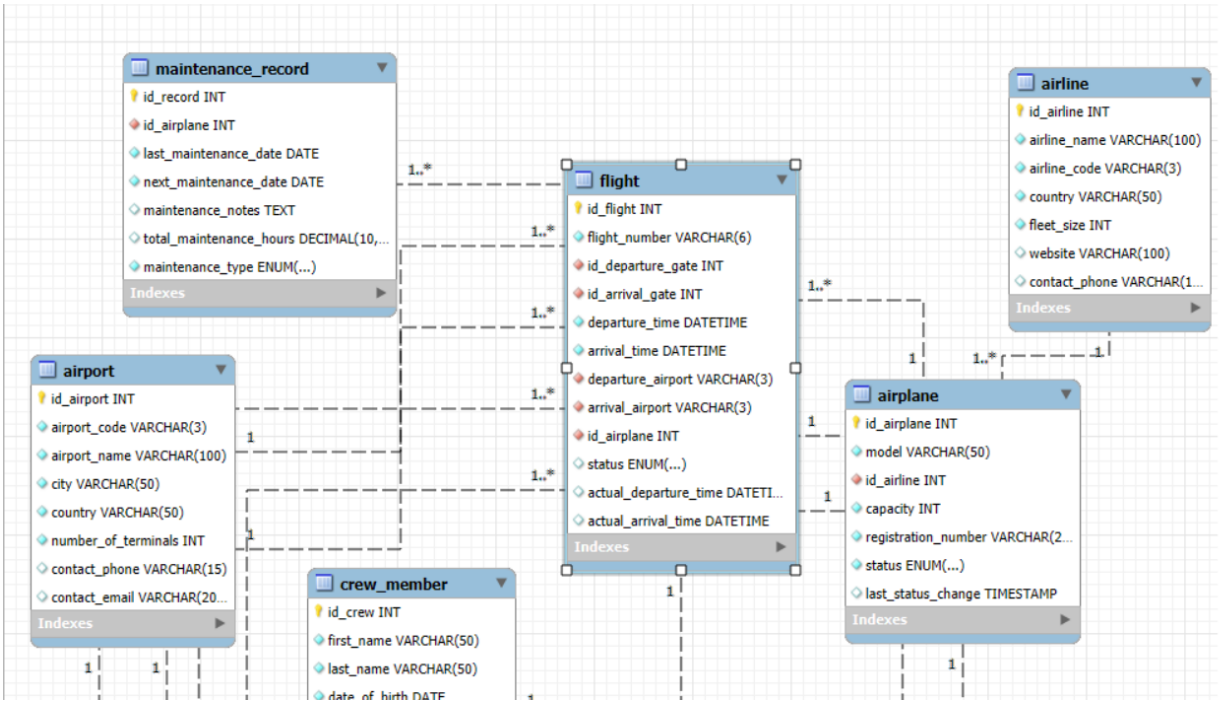
To provide abstraction, security, and maintaianbility, the design of the database system follows the three-level architecture that divides the database into internal, conceptual, and external levels. Internally, the system is the physical disk storage of information. This includes indexes, the optimization of the path used in retrieving data, and methods for storage. Although not externally visible, it is very important to dependability and performance.

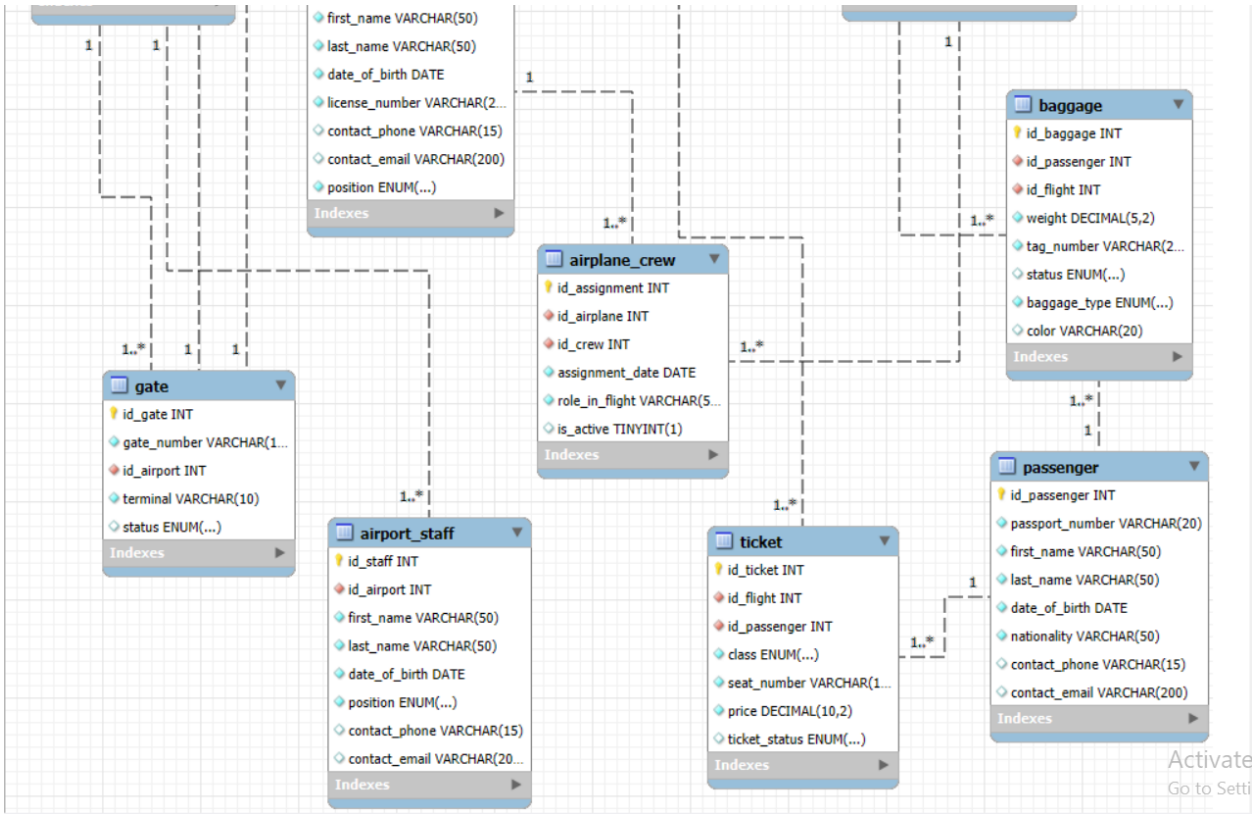
The logical structure of the database is described at the conceptual level. Each of the entities—airport, airplane, flight, passenger, ticket, baggage, and relationships between them—is defined. This level uses constraints, and foreign keys to enable data integrity. Regardless of how data is stored or accessed, everyone and every program talks to the system in terms of one interface — how it's logically connected and defined in the database.

The system has unique views for unique users at the exterior level as pictured in the second picture below. For instance, passengers can see information about their reservations, baggage, and flight status, airport personnel can see and manage information about flights, gates, and aircraft maintnance. By ensuring that each user accesses only the data that is related to their role, this division increases data security and user experience.

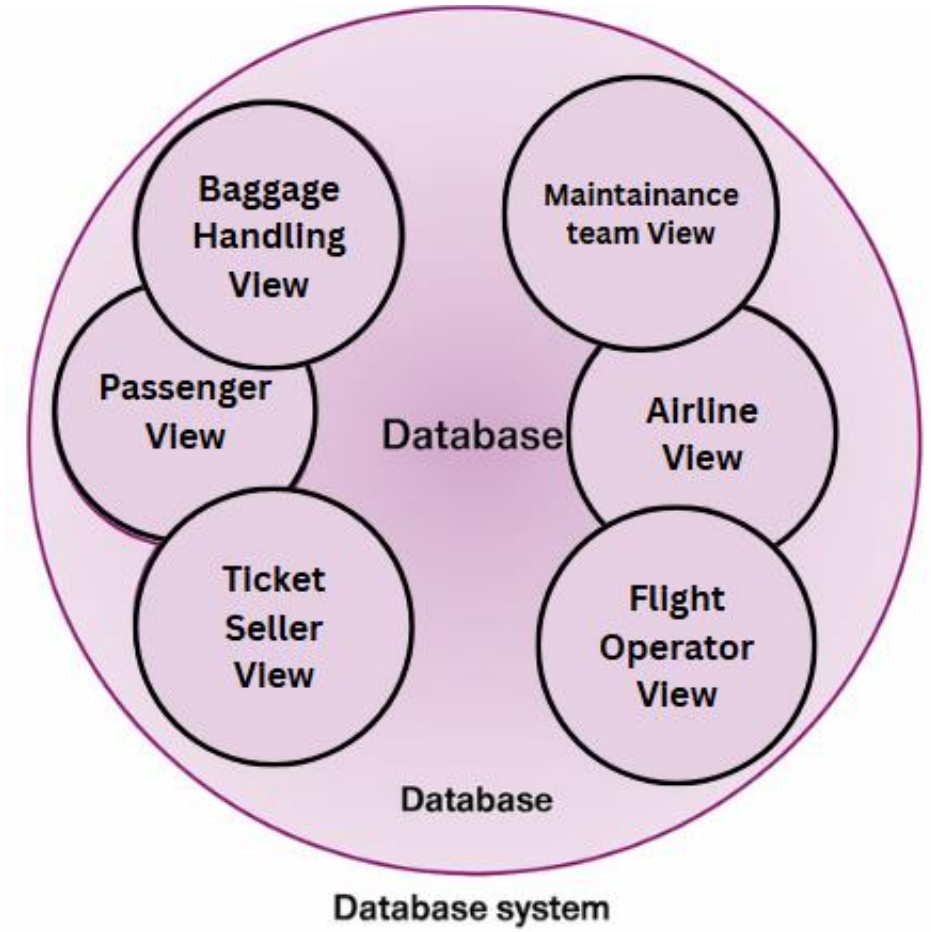
Smooth operations management at the airport, avoiding data redundancy, and ensuring same data integrity for all the entities are the top goals of the Airport Management System database. Aspects such as dashboards, analytics functions, and mobile apps in future add-ons are made possible due to the modularity and scaleability of the system. It further permits secure access using role-based access to data by views or application-layer permissions and provides end-user ease of use while maintaining secrecy of sensitive data.

The following picture will visually demonstrate how the database is connected, i.e. how the entities are connected. It also showcases, the entitiess attributes, primary keys, and types of relationships implemented. Everything will be explaind in more detail in the following parts of the report.





Picture 1: The Diagram of the database



Picture 2: Views diagram

2.2 Identify the purpose of the DBMS

The function of this database menagement system is to support the activities of an airport by storing and maintaning required information related to flights, passengers, airplanes, airlines, gates, employees, and luggage. The system focuses on improving efficiency, accuracy, and ease of information in the airport environment. The system allows central storage of information and consistency by means of constraints and relationships among tables. This DBMS will be used for tracking following aspects:

A) Flight scheduling and tracking – managing flight details, times, statuses, and gate assignments
B) Passenger management – storing personal information, contact details, and travel history
C) Airplane tracking – keeping records of airplane models, status, and airline ownership
D) Crew member assignments – assigning crew to airplanes and tracking their roles and activity
E) Ticketing – recording ticket purchases, seating, travel class, and ticket status
F) Baggage handling – managing baggage details, weight, tags, and current status

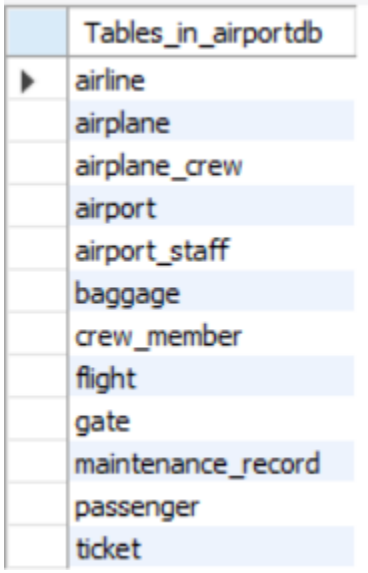
G) Airport staff management – keeping staff information organized by roles, terminals, and contact details
H) Maintenance records – storing airplane maintenance history and future scheduled checks

Overall, this system aims to deliver a flexible, secure, and user-friendly solution that meets both current and future demands of airport operations, while keeping sensitive data safe through role-based access control.

2.3 Determining the entities, types of attributes and relationships involved

2.3.1 Entities

The Airport Management System consists of several main entities that represent the central aspects of airport operations. These entities are airline, airplane, airplane crew, airport, airport staff, baggage, crew member, flight, gate, maintenance record, passenger and ticket. All of the entities listed are shown in Picture 2 from the database. Each entity plays a different role but all are conected, in the database to ensure accuracy.

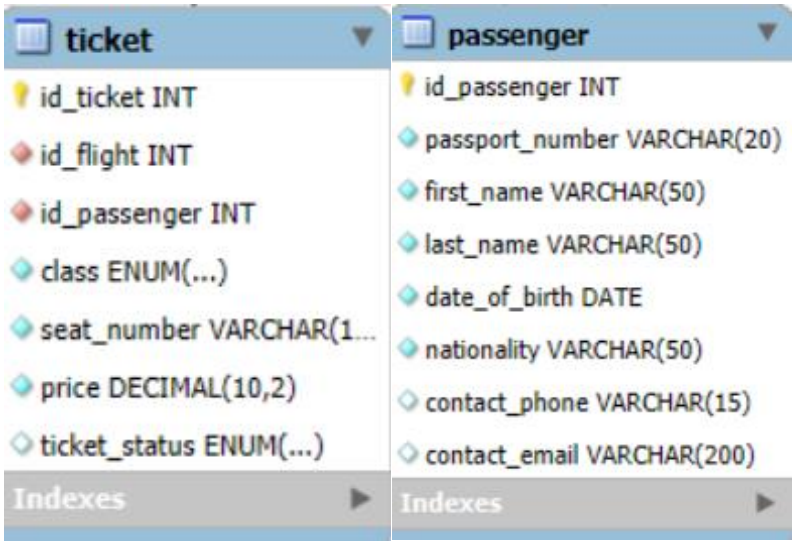


Tables_in_airportdb
airline
airplane
airplane_crew
airport
airport_staff
baggage
crew_member
flight
gate
maintenance_record
passenger
ticket

Picture 3: All the Entities in database

2.5.2 Attributes

All the entities contain specific attributes based on its role in the system. For example, the Passenger entity contains contact and personal information, whereas the Flight entity contains scheduling data and references to departure and arrival gates. (Picture 3)



Picture 4: Attributes of entities ticket and passenger

Each of the Airport Management System entities is defined by a set of attributes that contain specific information about the entity. These attributes have been selected with the necessity to precisely model real-world data and also to provide for the varied functionality of the system. Following is a list of all the attributes used in the different entities of the database:

A) String/Text Types (VARCHAR/CHAR)
   airport_code (VARCHAR(3))
   airport_name (VARCHAR(100))
   city (VARCHAR(50))
   country (VARCHAR(50))
   airline_name (VARCHAR(100))
   flight_number (VARCHAR(6))
   first_name (VARCHAR(50))
   last_name (VARCHAR(50))
   passport_number (VARCHAR(20))
   license_number (VARCHAR(20))
   model (VARCHAR(50))
   registration_number (VARCHAR(20))
   gate_number (VARCHAR(10))
   terminal (VARCHAR(10))
   seat_number (VARCHAR(10))
   tag_number (VARCHAR(20))
   color (VARCHAR(20))

contact_phone (VARCHAR(15))
contact_email (VARCHAR(200))
website (VARCHAR(100))
newspaper_name (VARCHAR(100))
bName (VARCHAR(100))

B) String/Text Types (VARCHAR/CHAR)
airport_code (VARCHAR(3))
airport_name (VARCHAR(100))
city (VARCHAR(50))
country (VARCHAR(50))
airline_name (VARCHAR(100))
flight_number (VARCHAR(6))
first_name (VARCHAR(50))
last_name (VARCHAR(50))
passport_number (VARCHAR(20))
license_number (VARCHAR(20))
model (VARCHAR(50))
registration_number (VARCHAR(20))
gate_number (VARCHAR(10))
terminal (VARCHAR(10))
seat_number (VARCHAR(10))
tag_number (VARCHAR(20))
color (VARCHAR(20))
contact_phone (VARCHAR(15))
contact_email (VARCHAR(200))
website (VARCHAR(100))
newspaper_name (VARCHAR(100))
bName (VARCHAR(100))

C) Numeric Types
number_of_terminals (INT)
fleet_size (INT)
capacity (INT)
weight (DECIMAL(5,2))
price (DECIMAL(10,2))
total_maintenance_hours (DECIMAL(10,2))

D) Date/Time Types
departure_time (DATETIME)
arrival_time (DATETIME)
actual_departure_time (DATETIME)
actual_arrival_time (DATETIME)
last_maintenance_date (DATE)
next_maintenance_date (DATE)
date_of_birth (DATE)
assignment_date (DATE)
last_status_change (TIMESTAMP)

E) Boolean Type
is_active (BOOLEAN)

F) ENUM Type
class (ENUM('Economy','Premium Economy','Business','First'))
status(ENUM('Scheduled','Boarding','Departed','In Air','Landed','Cancelled','Delayed'))
baggage_type (ENUM('Carry-on','Checked'))
baggage_status (ENUM('Checked','Loaded','Transferred','Arrived','Lost'))
position (ENUM('Pilot','Co-Pilot','Flight Attendant','Captain','First Officer'))
maintenance_type (ENUM('Routine Inspection','Minor Repair','Major Overhaul','Engine Service','Avionics Update','Structural Repair','Safety System Check'))
ticket_status (ENUM('Confirmed','Cancelled','Used'))

```
CREATE TABLE Airport (
    id_airport INT PRIMARY KEY AUTO_INCREMENT,
    airport_code VARCHAR(3) UNIQUE NOT NULL,
    airport_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    country VARCHAR(50) NOT NULL,
    number_of_terminals INT NOT NULL CHECK (number_of_terminals > 0),
    contact_phone VARCHAR(15) CHECK (
        contact_phone IS NULL OR
        (contact_phone REGEXP '^\\+[0-9]{7,15}$' AND
         contact_phone NOT LIKE '%[^0-9+]%')
    ),
```

Picture 5: Example of creating entity Airport and its attributes

2.2.3 Relationships

To ensure data consistency and reflect actual-world relationships among different components of the airport system, relationships were established using foreign keys. They define how the entities are interrelated and help ensure the data integrity. A list of all the relationships in the database follows:

One-to-Many (1:N) Relationships

- Airport to Gate: one Airport can have many Gates
- Airline to Airplane: one Airline can operate many Airplanes
- Airplane to Flight: one Airplane can be assigned to many Flights
- Flight to Ticket: one Flight can have many Tickets
- Passenger to Ticket: one Passenger can book many Tickets
- Passenger to Baggage: one Passenger can check many Baggage items
- Flight to Baggage: one Flight can carry many Baggage items
- Airport to Flight (Departure): one Airport can be the departure point for many Flights
- Airport to Flight (Arrival): one Airport can be the arrival point for many Flights
- Airport to Airport_Staff: one Airport can employ many Staff members

Many-to-Many (M:N) Relationships

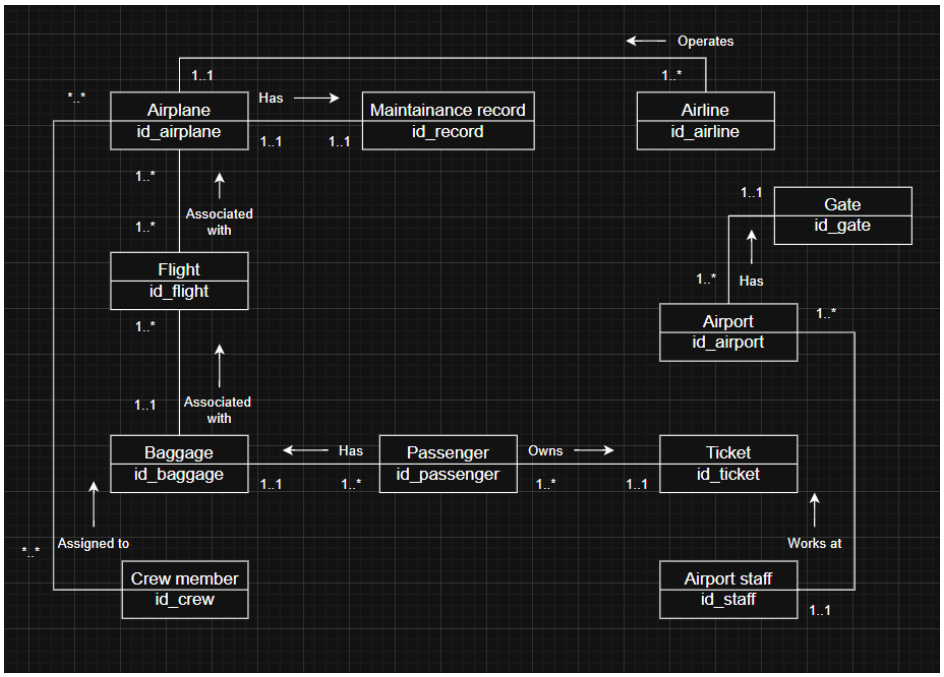
- Airplane to Crew_Member: one Airplane can have many Crew Members and one Crew Member can be assigned to many Airplanes. For this the junction table Airplane_Crew is used.

One-to-One (1:1) Relationships

- Maintenance_Record to Airplane: one Airplane has exactly one current maintenance record

The relationships enable the database to properly reflect actual airport operations and give confidence that data in various components is properly related.

Most of the relationships can be pictured in the ER diagram below. As everything else was explained previously.



Picture 6: ER Diagram of the database

2.2.4 Constraints

To ensure data accuracy, integrity, and reliability, several constraints were imposed throughout the database design. These constraints guarantee that entered data complies with certain rules and conditions and does not include invalid or inconsistent information. Below are some examples of constraints employed in the system:

a) Ensures airport codes are globally unique
airport_code VARCHAR(3) UNIQUE NOT NULL

b) Guarantees no duplicate passport numbers exist
passport_number VARCHAR(20) UNIQUE NOT NULL

c) Mandates at least one terminal per airport
number_of_terminals INT NOT NULL CHECK (number_of_terminals > 0)

d) Validates airline fleet size (0-1000 aircraft)
fleet_size INT NOT NULL CHECK (fleet_size >= 0 AND fleet_size <= 1000)

e) Phone number must be either NULL or in E.164 format (+[country code][number])

```
contact_phone VARCHAR(15) CHECK (
    contact_phone IS NULL OR
    (contact_phone REGEXP '^\\+[0-9]{7,15}$'
```

```
    AND contact_phone NOT LIKE '%[^0-9+]%')
)
```

f) Email must be either NULL or RFC-compliant

```
contact_email VARCHAR(200) CHECK (
  contact_email IS NULL OR
  (contact_email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$'
   AND contact_email LIKE '%@%.%'
   AND contact_email NOT LIKE '%..%')
)
```

g) Website must be NULL or contain valid URL structure

```
website VARCHAR(100) CHECK (
  website IS NULL OR
  website LIKE 'http%://%.%' OR
  website LIKE 'https%://%.%'
)
```

At the time of requirement analysis, the primary objective was to understand the requirements of all the stakeholders who become part of the operations of an airport. They were airport managers, airline staff, maintenance personnel, and passengers. The system needed to support a wide range of data like passenger information, flight timings, ticket booking, baggage monitoring, aircraft status, crew assignments, and gate assignments.

Secondly, it had to track operational data such as maintenance logs and personnel assignments. Flight reservation, check-in, baggage registration, crew assignment, and maintenance scheduling were identified as key processes the system would have to support. Administrative personnel, airport personnel, and airline operators would be the primary users of the system, depending on their need for speedy access to accurate, real-time data for day-to-day activity at the airport. Through this analysis, a clear framework of data entities, relationships, and constraints was created in order to meet the functional as well as the non-functional requirements of a modern airport management system.

2.3 Queries

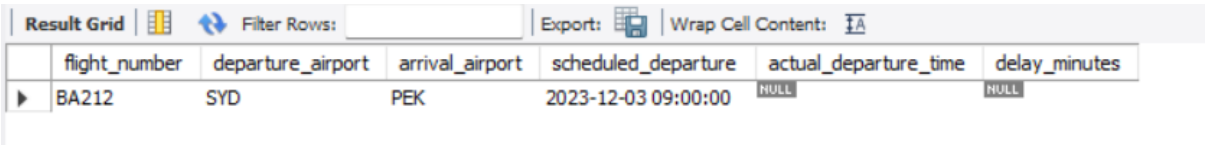
To show the practical use of the database, various SQL queries can be executed to retrieve valuable and relevant information from the system. These queries enable airport operations by providing insights, aiding in decision-making, and enabling efficient access to data. Some of the useful queries that can be used in the Airport Management System are listed below:

1. Delayed Flights: Find all delayed flights with their new estimated times

```sql
SELECT
    f.flight_number,
    a1.airport_code AS departure_airport,
    a2.airport_code AS arrival_airport,
    f.departure_time AS scheduled_departure,
    f.actual_departure_time,
    TIMESTAMPDIFF(MINUTE, f.departure_time, f.actual_departure_time) AS delay_minutes
FROM
    Flight f
JOIN
    Airport a1 ON f.departure_airport = a1.airport_code
JOIN
    Airport a2 ON f.arrival_airport = a2.airport_code
WHERE
    f.status = 'Delayed'
ORDER BY
    delay_minutes DESC;
```

Picture 7: Query Example 1

Output of given query:



flight_number	departure_airport	arrival_airport	scheduled_departure	actual_departure_time	delay_minutes
BA212	SYD	PEK	2023-12-03 09:00:00	NULL	NULL

Picture 8: Output of Query Example 1

2. Locate all baggage for a specific flight

```sql
SELECT
    b.tag_number, p.first_name, p.last_name,
    b.weight, b.baggage_type, b.status,
    CASE
        WHEN b.status = 'Loaded' THEN CONCAT('Flight ', f.flight_number)
        WHEN b.status = 'Arrived' THEN a.airport_name
        ELSE 'In transit'
    END AS location
FROM
    Baggage b
JOIN
    Passenger p ON b.id_passenger = p.id_passenger
JOIN
    Flight f ON b.id_flight = f.id_flight
LEFT JOIN
    Airport a ON f.arrival_airport = a.airport_code
WHERE
    f.id_flight = 52;
```

Picture 9: Query Example 2

Output of the given query:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tag_number	first_name	last_name	weight	baggage_type	status	location
DL101001	John	Smith	23.50	Checked	Arrived	Beijing Capital International Airport
DL101002	John	Smith	8.00	Carry-on	Arrived	Beijing Capital International Airport
DL101003	Emma	Johnson	18.00	Checked	Arrived	Beijing Capital International Airport
DL101004	Michael	Brown	25.00	Checked	Arrived	Beijing Capital International Airport

Picture 10: Output of Query Example 2

3. Generate a boarding pass for a specific passenger and flight

```sql
SELECT
    p.first_name, p.last_name, p.passport_number,
    f.flight_number, f.departure_time,
    t.seat_number, t.class,
    g.gate_number, a.airport_name AS departure_airport
FROM
    Ticket t
JOIN
    Passenger p ON t.id_passenger = p.id_passenger
JOIN
    Flight f ON t.id_flight = f.id_flight
JOIN
    Gate g ON f.id_departure_gate = g.id_gate
JOIN
    Airport a ON g.id_airport = a.id_airport
WHERE
    p.passport_number = 'A98765432' AND flight_number='BA212';
```

Picture 11: Query Example 3

Output of the given query:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name	passport_number	flight_number	departure_time	seat_number	class	gate_number	departure_airport
Emma	Johnson	A98765432	BA212	2023-12-03 09:00:00	10B	Economy	D12	Amsterdam Airport Schiphol

Picture 12: Output of Query Example 3

3 Conclusion

In summary, the Airport Management System database project provides a comperhensive solution to menage intricate and dynamic airport operations efficiently. The effective design and implementation of the database facilitate efficient management of a broad range of functions, such as flight scheduling and passenger management, baggage tracking, and maintenance recording. Also provides an insight in specific needs such as: finding all delayed flights with their estimated times, locating all the baggage for a certain flight, generating a flight ticket for a specific pessenger and specific flight, and a lot of other requests. Structured relationships, constraints, and normalized tables provide data integrity, minimize redundancy, and increase efficiency in operation.

This system not only serves as a centralized repository for critical airport data but also enables day-to-day planning and decision-making by retrieving precise real-time data. Via several SQL queries, the users can derive data and generate reports that assist in monitoring and improving the overall airport activity.

While the system is functional, there are future possibilities for enhancing the system, for instance, by expanding the capabilities of the system to handle more advanced scenarios, such as including real-time flight information, advanced security functions, and predictive maintenance data. Also additional needed entities should be added, along side with more attributes for each entity to provide more details and better description of the entity. In all, this database is an adequate foundation on which to modernize and facilitate airport management procedures, resulting in increased efficiency and customer service within the aviation industry.

4 Resources

https://tavtechnologies.aero/en-EN/review/pages/airporthistory
https://www.mysql.com
https://www.w3schools.com/mysql/
https://chatgpt.com
https://www.deepseek.com/
https://app.diagrams.net/

5 List of Figures

Picture 1: The Diagram of the database
Picture 2: Views diagram
Picture 3: All the Entities in database
Picture 4: Attributes of entities ticket and passenger
Picture 5: Example of creating entity Airport and its attributes
Picture 6: ER Diagram of the database
Picture 7: Query Example 1
Picture 8: Output of Query Example 1
Picture 9: Query Example 2
Picture 10: Output of Query Example 2
Picture 11: Query Example 3
Picture 12: Output of Query Example 3